

Task 1: Develop a C program that demonstrates how a Linux operating system executes a command entered by a user that 1. Accept a Linux command as input. 2. Create a child process using fork(). 3. Execute the command in the child process using an appropriate exec() system call. 4. Allow the parent process to wait for the child using wait (). 5. Display the Process ID (PID) of both parent and child processes.

- a) Write a C code to create a process using fork() system call.
- b) Read the user input as shell command.
- c) Use a system call exec() to execute the shell command.
- d) Use a wait() system call for waiting a parent process.
- e) Display the PIDs of parent and child processes.

Code:

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <unistd.h>
4  #include <sys/types.h>
5  #include <sys/wait.h>
6  #include <string.h>
7
8  int main()
9  {
10     char command[100];
11     pid_t pid;
12     int status;
13
14     // Read command from the user
15     printf("Enter a Linux command: ");
16     fgets(command, sizeof(command), stdin);
17
18     // Remove newline character
19     command[strlen(command) - 1] = '\0';
20
21     // Create child process
22     pid = fork();
23
24     if (pid < 0)
25     {
26         perror("fork failed");
27         return 1;
28     }
29
30     // Child process
31     if (pid == 0)
32     {
33         printf("\nChild Process:\n");
34         printf("Child PID : %d\n", getpid());
35         printf("Parent PID : %d\n", getppid());
36
37         // Execute command using shell
38         execl("/bin/sh", "sh", "-c", command, (char *)NULL);
39
40         // This executes only if execl() fails
41         perror("exec failed");
42         exit(1);
43     }
44
45     // Parent process
46     else
47     {
48         printf("\nParent Process:\n");
49         printf("Parent PID : %d\n", getpid());
50         printf("Child PID : %d\n", pid);
51
52         // Wait for child process to finish
53         wait(&status);
54
55         if (WIFEXITED(status))
56         {
57             printf("\nChild process finished with exit status: %d\n",
58                   WEXITSTATUS(status));
59         }
60     }
61
62     return 0;
63 }

```

OUTPUT:

```
Last login: Thu Sep 17 14:32:48 on console
green@kali:~/stress-MacBook-Pro - $ nano mnk.c
green@kali:~/stress-MacBook-Pro - $ gcc mnk.c -o mnk
green@kali:~/stress-MacBook-Pro - $ ./mnk
Enter a Linux command: ls

Parent Process:
Parent PID : 978
Child PID : 979

Child Process:
Child PID : 979
Parent PID : 978

98.c      2      34.c      bb.c      gh.c      Music      ss
1.c.save  2.c     36.c.save bh.c      helloworld  n.c.save  task
180.c     22.c   390.c     c         h.c       nano.c.save task.c
181.c     222.c 4.c       cc         jw.co     rnk.c.save th.c
111.c     2222.c 444.c     cc-c       k.c       NewFolderName tt.c.save
1111.c    23     444.c     og-c       k.c       mnk.c       v
11111.c   23.c   45.c     COW       k.c.save  Parallels   x.c
111111.c  24     486      cv         k.c.save  Pictures     we
1111111.c 24.c   486.c    cv-c       k.c       Parallels   we.c
11111111.c 243    487.c    data       k.c       process.c   week3
12.c      26     567.c    ddd        kn.c      process.c.save week2
123.c     26.c  567.c    ddd        kn.c      PROJECT_GSA 2 week2.c
123.c.save 264    66       Desktop   kn.c      projects     week2skill.c
1234      3       66.c     dir2       l.c       pu.c        week3
1234.c     3.c    79.c.save dir3       ll.c.save  Public      week2skill.c
12344.c    31.c   8.c      Documents ll.c       pwd         mw.c
123444.c   333.c 85        Downloads llll.c     pwd.c       python
1234444.c  333.c 85.c     file.c.save m.c       mn.c.save   q
12344444.c 334   Applications Applications (Parallels) ggm        mn.c.save   q.c
123444444.c 334.c Applications (Parallels) ggm        Movies      source.txt
18.c       34     88        ggm.c

Child process finished with exit status: 0
green@kali:~/stress-MacBook-Pro - $
```

Task-2: Using Linux terminal commands (uname, lscpu, ps, top), investigate the relationship between hardware resources and operating system services. Prepare a report explaining how the OS abstracts CPU, memory, storage, and I/O devices.

```
Linux FICD 5.09 File: 51.c

#include <stdio.h>
#include <stdlib.h>

int main()
{
    printf("=====n");
    printf(" LINUX HARDWARE & OS RESOURCE ANALYSISn");
    printf("=====n");

    // 1. Display OS and Kernel information
    printf("1. OPERATING SYSTEM INFORMATION (uname)n");
    printf("=====n");
    system("uname -a");

    // 2. Display CPU information
    printf("2. CPU INFORMATION (lscpu)n");
    printf("=====n");
    system("lscpu");

    // 3. Display running processes
    printf("3. RUNNING PROCESSES (ps)n");
    printf("=====n");
    system("ps aux");

    // 4. Display real-time CPU and memory usage
    printf("4. REAL-TIME SYSTEM MONITORING (top)n");
    printf("=====n");
    printf("Starting top command...n");
    printf("Press 'q' to exit top.n");

    system("top");

    printf("=====n");
    printf(" ANALYSIS COMPLETEDn");
    printf("=====n");

    return 0;
}
```

OUTPUT:

[illegible]